hashlock.

Security Audit

Cryptaine (DeFi)

April 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

Cryptaine partnered with Hashlock to conduct a security audit of their CRY Token.sol and CRY Token_flattened.sol contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Cryptaine is the world's first blockchain-powered affiliate marketing protocol, transforming traditional affiliate systems into a decentralized, trustless ecosystem. Leveraging smart contracts and on-chain USDC settlements, Cryptaine eliminates excessive fees (up to 30% on legacy platforms) and payment delays by enabling instant, immutable payouts. Its native $CRY token underpins a dynamic staking model—holders unlock tiered fee discounts (from 5% down to 1%) while gaining governance rights and fraud-resistant attribution through transparent, blockchain-verified tracking. With seamless plugin integration, real-time analytics, and a vibrant, DAO-like community driving platform growth, Cryptaine bridges Web2 affiliate networks with DeFi-style incentives, delivering a fair, efficient, and community-governed affiliate marketplace.

Project Name: Cryptaine

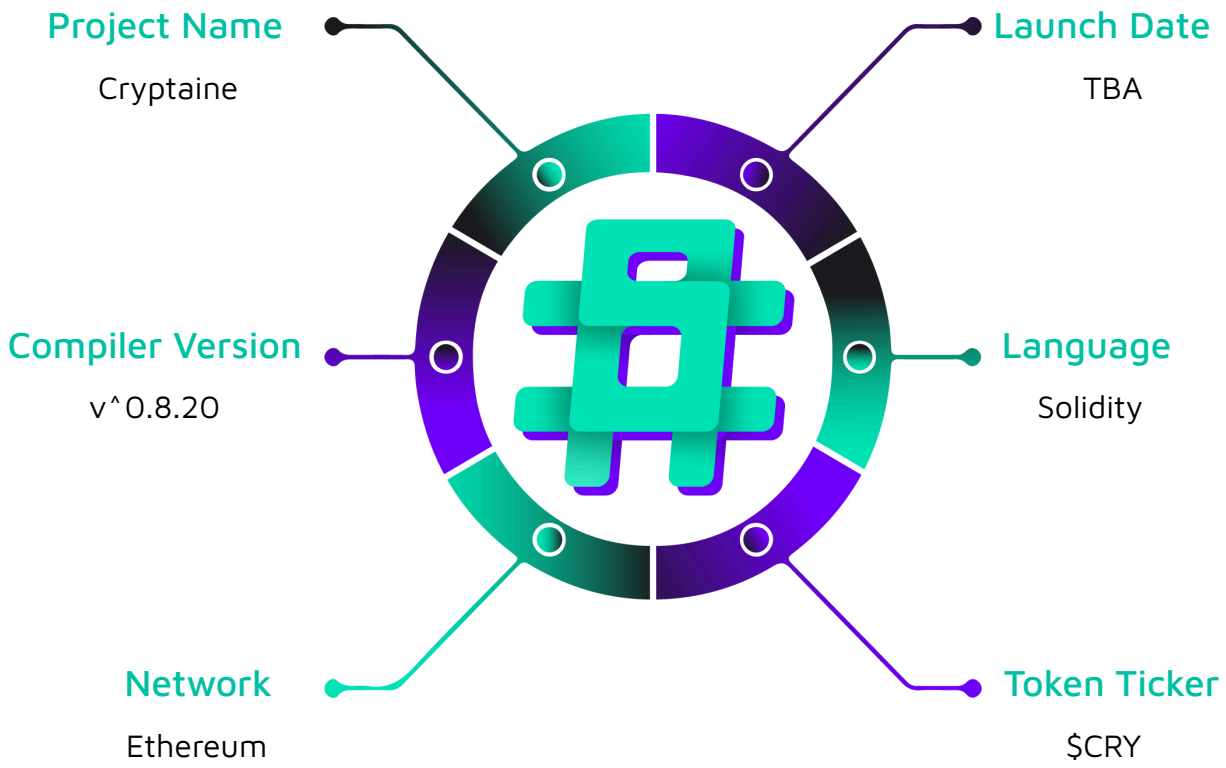
Project Type: Token

Compiler Version: ^0.8.20

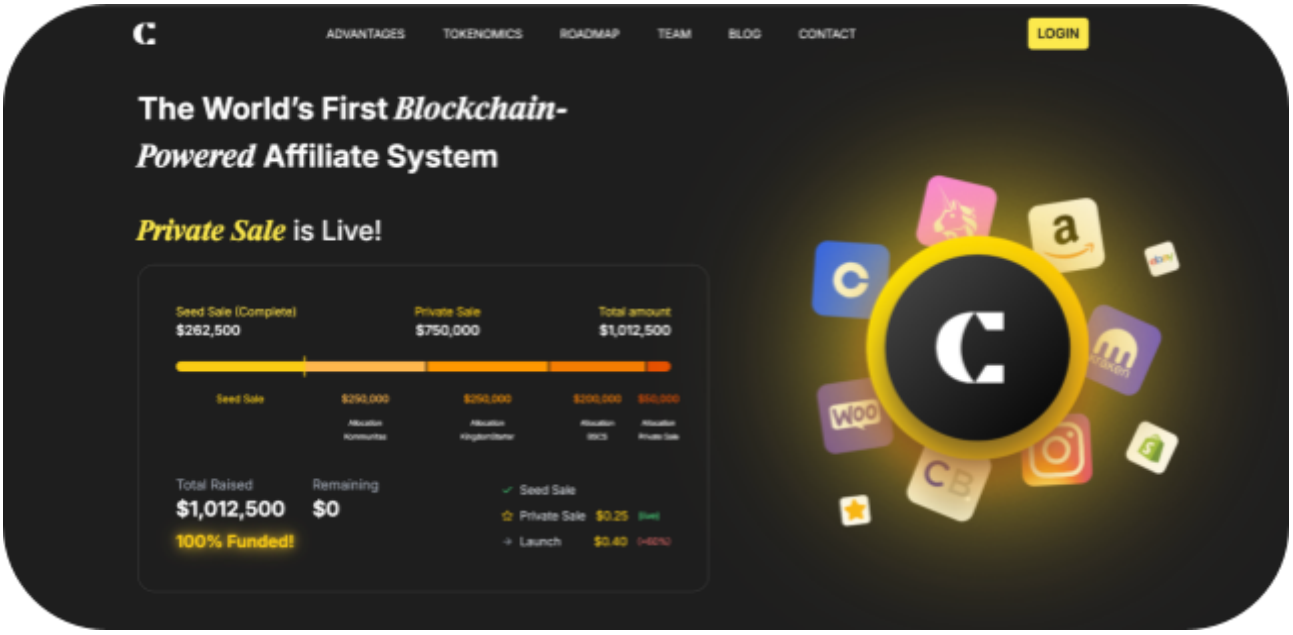
Website: https://cryptaine.com/

Logo:

Visualised Context:



Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Cryptaine, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Cryptaine Smart Contracts**
Platform	**Ethereum / Solidity**
Audit Date	**April, 2025**
Contract 1	CRY Token.sol
Contract 1 MD5 Hash	05a1e122a85cd8e8ccd68c5c081a09e2
Contract 2	CRY Token_flattened.sol
Contract 2 MD5 Hash	7682f148d0b0d4a968a447e5b3c8ea40
Audited GitHub Commit Hash	a3c1e6100192f078b74024197dfaf34755af0bdd
Fix Review GitHub Commit Hash	2be4d6bfdbfb8d158e9cd3107201e1ef753bf58e

Note: *Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.*

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 Low severity vulnerabilities

1 Gas Optimisations

3 QAs

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
CRY Token.sol - Allows users to: - Send tokens to another address. - Authorize another address to spend tokens. - Transfer tokens on behalf of approved spender. - Destroy their own tokens via burn() and burnFrom() (from ERC20Burnable). - Perform gasless approvals using off-chain signatures.	**Contract achieves this functionality.**
CRY Token_flattened.sol Flattened version of CRY Token.sol contract which contains all the periphery contracts within a single contract.	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the Cryptaine, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Cryptaine smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Low

[L-01] CRY Token, CRY Token_flattened - Unrestricted `renounceOwnership()` Call

Description

The contract inherits from OpenZeppelin's `Ownable` contract, which includes the `renounceOwnership()` function. This function allows the current owner to irreversibly relinquish ownership, setting the `owner` address to `address(0)`.

Impact

In the current implementation, `renounceOwnership()` is not overridden or restricted, meaning any accidental or intentional call by the owner will permanently remove owner control over the contract.

Recommendation

If ownership should never be renounced (common for fixed-governance or permanently-administered tokens), override and disable It by adding the following function:

```
function renounceOwnership() public override onlyOwner {

    revert("Renouncing ownership is disabled.");

}
```

Status

Resolved

[L-02] CRY Token, CRY Token_flattened - Use `Ownable2Step` contract rather than `Ownable` contract

Description

`Ownable` contract from OpenZeppelin is used in both `CRY Token.sol` and `CRY Token_flattened.sol` contracts. The primary concern with the `Ownable` contract's `transferOwnership` function is its immediacy and lack of verification. Once called, ownership is instantly transferred to the specified address without any confirmation from the recipient.

Impact

If the new owner's address is entered incorrectly, the contract's control could be lost or handed over to an unintended entity, leading to potential security risks or loss of functionality.

Recommendation

To mitigate this risk, it's advisable to use OpenZeppelin's `Ownable2Step` contract, this contract enhances the ownership transfer process by implementing a two-step mechanism.

This two-step process ensures that ownership is only transferred if the new owner explicitly accepts it, thereby preventing accidental transfers to incorrect addresses.

Status

Resolved

Gas

[G-01] CRY Token, CRY Token_flattened - Code Optimisation in `constructor`

Description

In `CRY Token::constructor`, the contract calculates `25000000 * 10 ** decimals()` twice, which increases the contract deployment cost.

Recommendation

The code can be refactored as follows:

```
constructor(

        address initialOwner

    ) ERC20("CRYPTAINE", "CRY") Ownable(initialOwner) ERC20Permit("CRYPTAINE") {

        MAX_TOTAL_SUPPLY = 25000000 * 10 ** decimals(); // Defines the max total supply.

--        _mint(msg.sender, 25000000 * 10 ** decimals()); // Mints initial supply.

++        _mint(msg.sender, MAX_TOTAL_SUPPLY); // Mints initial supply.



    }
```

Status

Resolved

QA

[Q-01] Contracts - Floating pragma

Description

The contracts `CRY Token.sol` and `CRY Token_flattened.sol` use a floating pragma version (^0.8.20). It's crucial to compile and deploy contracts with the same compiler version and settings used during development and testing. Fixing the pragma version prevents compilation with different versions. Using an outdated pragma version could introduce bugs that negatively affect the contract system, while newly released versions may have undiscovered security vulnerabilities.

Recommendation

Change the pragma statement in both contracts to a fixed version, such as pragma

solidity 0.8.26. This locks the contract to a specific compiler version.

Status

Resolved

[Q-02] CRY Token, CRY Token_flattened - Max Total Supply Comment Mismatch

Description

The comments about the declaration of `MAX_TOTAL_SUPPLY` in `CRY Token.sol::Line 15` and `CRY Token_flattened.sol::Line 2291` states that `Max supply capped at 100 million tokens` while the value is actually set to `25000000 * 10 ** decimals()`, which is equivalent to 25 million tokens (decimal is 18).

Recommendation

Update the comments (or the value) so that it accurately reflects the intended maximum total supply.

Status

Resolved

[Q-03] CRY Token, CRY Token_flattened - Inheriting OpenZeppelin's `Ownable` contract but the `onlyOwner` modifier is not used anywhere

Description

The `Ownable` contract establishes an ownership model, providing the `onlyOwner` modifier to restrict access to specific functions. However, this modifier is not used anywhere in `CRY Token.sol` and `CRY Token_flattened.sol` contracts.

Recommendation

To address this issue, identify functions that should be restricted to the contract owner and apply the `onlyOwner` modifier to them. Otherwise, remove the `Ownable` contract if it is not in use.

Status

Resolved

Centralisation

Cryptaine values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, Cryptaine seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au